Common Python Mistakes: A Patient Beginner's Guide

Learning Python involves learning how a computer follows instructions, not memorising commands. Mistakes are useful evidence about your current mental model. Before asking AI for a correction, predict what your program will do, run it, and compare the result with your prediction. Keep a small mistake diary containing the symptom, its cause, the corrected example, and one test. The ten mistakes below connect everyday beginner confusion with a practical repair. These examples use Python 3 and deliberately avoid advanced shortcuts so that you can explain each step yourself.

1. Treating input as a number

Python's input function returns text. A learner may type twenty and expect arithmetic automatically. Even when the characters look numeric, the value remains a string until converted. Adding two strings joins their characters rather than adding their numeric values. Convert deliberately, and remember that invalid numeric text can raise ValueError. Try both a normal number and an invalid entry rather than checking only the happy path.

```
age_text = "18"
age = int(age_text)
print(age + 1)  # 19
```

2. Confusing assignment with comparison

A single equals sign assigns a value to a name. Two equals signs compare values and produce a boolean result. Assignment is not a mathematical statement that stays true forever: a later assignment changes the binding. Read assignments as "now refers to" rather than "is always equal to." Explain each line in execution order before running it.

```
score = 40
print(score == 40)  # True
score = score + 1
print(score)  # 41
```

3. Ignoring indentation and colons

Indentation is part of Python's structure, not decorative spacing. A colon introduces a block after statements such as if, for, and function definitions. Use consistent spaces, usually four per level, and avoid mixing tabs with spaces. When indentation fails, inspect the surrounding block instead of changing random lines. Ask which statements should execute together and which should run afterward.

```
score = 65
if score >= 40:
    print("Pass")
print("Finished")
```

4. Expecting range to include its stop

The stop argument of range is excluded. This convention works naturally with zero-based indexing but surprises learners counting from one. Write down the exact sequence before using it in a calculation. If you want integers one through five, the stop must be six. Check a small boundary case; a loop that looks correct may silently skip the final value.

```
total = 0
for number in range(1, 6):
    total += number
print(total)  # 15
```

5. Accessing a list outside its boundaries

The first element has index zero, so the final valid positive index is length minus one. Accessing index len(items) raises IndexError. Empty lists have no first element at all. Prefer direct iteration when you need values rather than positions. Negative indexing can select the final element, but only when the collection contains something. Test an empty list explicitly.

```
items = [10, 20]
for item in items:
    print(item)
if items:
    print(items[-1])  # 20
```

6. Mistaking print for return

Printing displays information; returning provides a result to the caller. A function without an explicit returned value produces None. If a later calculation needs the answer, return it and print outside the function. This separation makes testing easier because an assertion can compare returned values directly. Trace the caller and function together to see where control goes next.

```
def double(number):
    return number * 2

result = double(4)
print(result + 1)  # 9
```

7. Accidentally sharing the same list

Assigning one list variable to another does not make a separate list. Both names can refer to the same mutable object. Appending through either name changes that object. For a flat list, copy creates an independent outer list. This is a shallow copy, so nested mutable objects would still need additional care. Do not generalise this example to every structure.

```
original = [1, 2]
copied = original.copy()
copied.append(3)
print(original)  # [1, 2]
```

8. Forgetting to update a while condition

A while loop repeatedly checks its condition. If nothing changes that condition, execution may continue indefinitely. Identify the initial state, continuation rule, and progress step before running the loop. Trace two iterations by hand. If a program becomes stuck, interrupt it and inspect the update rather than repeatedly restarting it. Progress toward termination should be visible.

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

9. Catching every error indiscriminately

A bare except can hide defects unrelated to the input you intended to validate. Catch the exception you expect and keep the protected block small. When conversion fails, tell the user what kind of input is required. Do not replace conversion with eval: executing arbitrary input is unnecessary and unsafe for this task. Unexpected errors deserve investigation, not silent suppression.

```
try:
    number = int("oops")
except ValueError:
    print("Enter an integer.")
```

10. Trusting a convincing AI solution without testing

An explanation can sound patient and still contain a wrong boundary, invented function, or mismatched requirement. Start with the problem contract and compare normal, boundary, and invalid cases. Ask AI for a hint before requesting full code. Then explain the corrected program without looking. Never enter passwords, private student records, or confidential assessment questions into a public tool. Credit assistance honestly.

```
from src.solutions import sum_to_n
assert sum_to_n(0) == 0
assert sum_to_n(3) == 6
```

Your goal is a repeatable debugging habit: predict, run, inspect, repair, and retest. One successful example does not prove general correctness. Small checks and clear explanations make progress visible and reduce dependence on copied answers.

When revising, change one assumption at a time and record what the new output teaches you. Explain the result to a classmate before attempting a similar problem independently. When revising, change one assumption at a time and record what the new output teaches you. Explain the result to a classmate before attempting a similar problem independently. When revising, change one assumption at.